Guide d'Étude Structuré : Architecture et Terminologie `JavaScript`

Niveau 1 : Genèse, Écosystème et Outils

Les origines du langage, sa standardisation et les environnements de travail.

- **`JavaScript`** : Le langage de programmation asynchrone, multiparadigme et basé sur des prototypes. Il permet de créer des éléments interactifs et de traiter des informations côté client (navigateur) et côté serveur.
- **Historique et Standardisation** :
  - **`Brendan Eich`** & **`Netscape`** : `Brendan Eich` est l'ingénieur ayant créé `JavaScript` (initialement chez `Netscape`, l'entreprise pionnière des navigateurs Web) pour rendre les pages interactives.
  - **`ECMAScript`** & **`TC39`** : `ECMAScript` est la spécification officielle dictant les règles universelles du langage. Le `TC39` est le comité d'experts mondiaux responsable de son évolution et de sa standardisation.
- **Intégration** :
  - **`Script Web`** : Le bloc de code source intégré directement dans le squelette de la page Web pour être compilé et exécuté par le navigateur.
- **Outils de Développement** :
  - **`IDE` (Integrated Development Environment)** : Un logiciel spécialisé orchestrant l'écriture, l'analyse et le débogage du code.
  - **Éditeurs majeurs** : **`Visual Studio Code`** (plateforme universelle extensible par Microsoft), **`WebStorm`** (infrastructure puissante et propriétaire par JetBrains), **`Sublime Text`** (éditeur ultra-rapide agissant en bas niveau), **`Vim`** (éditeur historique en ligne de commande fonctionnant sans souris).
  - **`MDN Web Docs`** : L'encyclopédie de référence maintenue par Mozilla, documentant les standards du Web et les API des navigateurs.

Niveau 2 : Primitives, Variables et Fondamentaux

Les types de données de base, la gestion de la mémoire et la logique d'évaluation.

- **Typage et Conventions** :
  - **`Data types`** : Les catégories fondamentales d'informations (texte, nombres, listes) reconnues par l'interpréteur.
  - **`Dynamic typing`** & **`Weak typing`** : Le langage associe le type à la valeur au moment de l'exécution et autorise des conversions implicites (typage faible), permettant une grande flexibilité mais exigeant de la rigueur.
  - **`camelCase`** : Convention typographique standard consistant à écrire les mots composés sans espaces, en mettant une majuscule à chaque mot sauf le premier (ex: `maVariableExemple`).
- **Déclaration de Variables** :
  - **`var`** : L'ancienne méthode de déclaration (obsolète), souffrant de problèmes de fuites en raison de son absence de limite de bloc.

  - `let` : Déclare une variable réassignable dont la portée est strictement confinée au bloc courant.
  - `const` : Déclare une référence immuable. La valeur primitive ne peut être réassignée, sécurisant ainsi l'intention algorithmique.
- **Valeurs Primitives** :
  - `String` : Séquence de texte immuable encodée en UTF-16.
  - `Boolean` : État logique absolu (`true` ou `false`).
  - **Évaluations implicites** : `Truthy values` (considérées comme vraies dans une condition) et `Falsy values` (strictement limitées à `false`, `0`, `""`, `null`, `undefined`, `NaN`).
  - `Undefined` : L'état par défaut d'une variable déclarée mais non initialisée.
  - `Null` : L'absence structurelle absolue d'une valeur, assignée intentionnellement par le développeur.
- **Nombres et Mathématiques** :
  - `BigInt` : Type primitif spécial pour manipuler des nombres entiers gigantesques dépassant les limites standards.
  - `NaN` **(Not a Number) &** `isNaN` : Signale l'échec d'une opération mathématique. `isNaN` permet de vérifier si une valeur a été corrompue lors d'un calcul.
  - `Infinity` : Représente l'asymptote mathématique (ex: division par zéro).
- `typeof` : Opérateur permettant de vérifier dynamiquement la classification primitive d'une variable avant son utilisation.

Niveau 3 : Opérateurs et Flux de Contrôle

Diriger l'exécution du programme via la logique conditionnelle et les itérations.

- **Opérateurs Logiques et Conditionnels** :
  - `Logical operators` **(`&&`, `||`, `!`)** : Les portes logiques ET, OU, NON. Elles utilisent souvent l'évaluation en `Court-circuit` **(`Short-circuit evaluation`)**, arrêtant l'analyse dès que le résultat est connu.
  - `Ternary operator` : Syntaxe compressée conditionnelle : `(condition ? valeurSiVrai : valeurSiFaux)`.
  - `Nullish coalescing (??)` : Opérateur renvoyant la valeur de droite uniquement si la valeur de gauche est strictement `null` ou `undefined`.
  - `Optional chaining (?.)` : Permet de lire profondément dans un objet complexe sans déclencher d'erreur si un maillon est manquant.
  - `increment / decrement` **(`++`, `--`)** : Raccourcis pour ajouter ou soustraire une unité à une variable.
- **Structures de Contrôle** :
  - `if...else` : Le carrefour décisionnel fondamental (si/sinon).
  - `switch` : Structure centralisée remplaçant de multiples `if...else` lorsqu'une variable peut prendre plusieurs valeurs précises.
  - `while` **&** `do...while` : Boucles se répétant tant qu'une condition est vraie. `do...while` garantit au moins une première exécution avant de vérifier la condition.
  - `continue` : Force une boucle à ignorer le traitement de l'élément actuel et à sauter immédiatement à l'itération suivante.

Niveau 4 : Fonctions et Mécaniques de Portée

Encapsulation de la logique, gestion du contexte et paradigmes fonctionnels.

- **Mécaniques d'Exécution** :
  - `Scope` / `Variable scope` : L'espace d'évaluation lexical délimitant la durée de vie et la visibilité d'une variable au sein du programme.
  - `Hoisting` : Mécanisme interne hissant (virtuellement) les déclarations de fonctions et de variables au sommet de leur portée avant l'exécution.
- **Structure Fonctionnelle** :
  - `Parameters` **(Paramètres)** : Variables d'entrée préparées dans la signature de la fonction pour recevoir des données externes.
  - `return` : Instruction stoppant immédiatement la fonction et renvoyant le résultat au contexte qui l'a appelée.
- **Concepts Avancés** :
  - `Closure` : Fonction mémorisant les variables de la portée dans laquelle elle a été créée, même après la fin d'exécution de son parent.

```
function createCounter() {
  let count = 0;
  return function() { count++; return count; };
}
```

  - `Lambda` / `Fonctions fléchées (Arrow functions)` : Syntaxe moderne et concise pour des fonctions anonymes. Elles conservent automatiquement le contexte lexical global (`this`).
  - `Callback` : Fonction transmise en tant qu'argument à une autre fonction, destinée à être invoquée plus tard (par exemple, à la fin d'une tâche).

Niveau 5 : Structures de Données et Collections

Gérer et manipuler des groupes complexes d'informations.

- **Conteneurs de Base** :
  - `Object` : Entité fondamentale regroupant des propriétés (clés-valeurs) et des méthodes. Il sert de modèle au format de transfert textuel universel `JSON`.
  - `Array` & `Collections indexées` : Listes séquentielles numérotées permettant le stockage ordonné d'éléments arbitraires.
  - `Map / Set` **(Keyed collections)** : Structures performantes. `Map` agit comme un dictionnaire acceptant n'importe quel type de clé. `Set` est une collection interdisant strictement les doublons.
- **Itération (Boucles spécialisées)** :
  - `Itérable` : Toute structure pouvant être parcourue séquentiellement.
  - `for...in` : Parcourt manuellement toutes les clés d'un objet (à éviter pour les tableaux).
  - `for...of` : La boucle moderne pour lire séquentiellement les valeurs d'une collection (comme un `Array`).

    - `forEach` : Méthode de tableau distribuant une fonction de rappel (`Callback`) pour chaque élément.
- **Manipulation d'`Array` et `String`** :
    - `split` : Découpe une chaîne de texte en un tableau, basé sur un séparateur.
    - `join` : Assemble les éléments d'un tableau en une seule chaîne de texte.
    - `slice` : Copie une portion ciblée d'un tableau ou texte sans modifier l'original.
    - `splice` : Outil destructeur amputant, greffant ou remplaçant des éléments directement dans le tableau d'origine.
    - `includes` : Vérifie simplement si une valeur particulière existe dans une liste ou un texte.
- **Syntaxe ES6+** :
    - `Destructuring` : Extrait directement des valeurs d'un tableau ou objet pour les assigner à des variables distinctes (`const { id, nom } = utilisateur;`).
    - `Opérateur rest (...)` : Regroupe un nombre indéfini d'arguments restants dans un tableau unique.
- **Utilitaires Globaux** :
    - `Math object` : Fournit les constantes et outils de calcul complexes (arrondis, cryptographie primitive).
    - `Date object` : Outil intégré capturant, manipulant et formatant des informations liées au temps.

Niveau 6 : Le `DOM` et le Modèle Objet du Navigateur

Interagir avec l'environnement du navigateur et modifier visuellement la page Web.

- **Modèles et Interfaces** :
    - `BOM` **(Browser Object Model)** : Outils fournis par le navigateur (historique, dimensions de l'écran, barre d'adresse) en dehors de la page elle-même.
    - `DOM` **(Document Object Model)** : La représentation en arborescence de la structure `HTML` de la page, manipulable en temps réel par `JavaScript`.
- **Hiérarchie Arborescente** :
    - `Node` : La plus petite brique du `DOM` (texte, commentaire, ou balise).
    - `Element` & `HTMLElement` : Représentation technique d'une balise `HTML` spécifique, possédant des propriétés de surface et de style.
    - `NodeList` : Une liste indexée d'éléments récupérés lors d'une recherche sur la page.
- **Recherche dans le `DOM` (`DOM retrieval`)** :
    - `querySelector` : Trouve le tout premier élément correspondant à une règle CSS (classe, ID).
    - `querySelectorAll` : Collecte sous forme de liste tous les éléments correspondant à une recherche.
    - `children` : Propriété exposant uniquement les balises `HTML` directement contenues dans un élément.
- **Manipulation du `DOM` (`DOM manipulation`)** :
    - `append` & `appendChild` : Insèrent dynamiquement de nouveaux éléments ou textes à la fin d'un conteneur parent.
    - `className` & `classList` : `className` réécrit intégralement les classes CSS de l'élément, tandis que `classList` permet un contrôle fin (ajouter, retirer).
    - `innerHTML` : Remplace ou lit intégralement le code `HTML` à l'intérieur d'un élément (attention aux failles de sécurité).

  - `textContent` : Extrait ou remplace de manière hautement sécurisée uniquement le texte pur, en ignorant les balises.
- **Interactions et Débogage** :
  - `alert`, `prompt`, `confirm` : Boîtes de dialogue natives bloquant l'exécution de la page pour informer ou questionner l'utilisateur.
  - `Console` : Tableau de bord invisible utilisé par les développeurs pour afficher des journaux et diagnostiquer le programme.

Niveau 7 : Événements et Interactivité des Formulaires

Capter les actions de l'utilisateur et valider les données.

- **Le Système d'Événements (`Events`)** :
  - `Events` : Signaux (comme `click` ou `dblclick`) émis par le système pour notifier qu'une action s'est produite.
  - `EventTarget` : Le moule de base que possède tout élément capable de réagir à des actions.
  - `Event Listeners` **(`addEventListener`, `removeEventListener`)** : Les auditeurs invisibles attachés à un élément pour écouter un événement et exécuter un bloc de code, ou s'en détacher pour libérer la mémoire.

```
  bouton.addEventListener("click", () => console.log("Bouton
cliqué"));
```

- **Propagation des Événements** :
  - `Propagation` & `Event Bubbling` : Le voyage naturel d'un événement qui remonte d'un élément enfant vers ses éléments parents.
  - `stopPropagation` : Empêche l'événement de remonter, l'étouffant à sa source.
  - `stopImmediatePropagation` : Bloque non seulement la remontée, mais empêche également les autres écouteurs attachés au même élément de s'exécuter.
  - `preventDefault` : Annule le comportement natif du navigateur (ex: empêcher un lien de changer de page ou un formulaire de recharger la page).
- **Formulaires et Validation** :
  - `FormData` : Compile automatiquement les données d'un formulaire dans un format prêt à être expédié par le réseau.
  - `Validation des contraintes API` & `setCustomValidity` : Le système du navigateur vérifiant la validité des champs avant envoi. `setCustomValidity` permet d'imposer un message d'erreur sur mesure.

Niveau 8 : Modèles Asynchrones et Boucle d'Événements

Gérer la concurrence, les minuteries et les appels réseaux sans bloquer l'application.

- **Architecture Concurrente** :

  - `Event Loop` : L'orchestrateur de trafic unithread qui s'assure que les opérations longues (réseau, minuteries) n'interrompent pas les tâches immédiates de l'interface.

- **Minuteries (`Timers`)** :
  - `Timeout` **(`setTimeout`)** : Un compte à rebours déclenchant une fonction une seule et unique fois après un délai donné.
  - `Interval` **(`setInterval`)** : Une boucle répétant inlassablement une action à rythme régulier jusqu'à arrêt explicite.

- **Promesses (`Promises`)** :
  - `Promises` : Contrats asynchrones garantissant un résultat futur (succès ou échec).
  - `resolve / reject` : Les leviers scellant le sort d'une promesse (succès ou erreur critique).
  - **Combinaisons de Promesses** :
  - `Promise.all` : Exécute plusieurs promesses et attend que *toutes* réussissent (échoue si une seule plante).
  - `Promise.allSettled` : Exécute plusieurs promesses et dresse un rapport final de ce qui a réussi ou échoué.
  - `Promise.any` : Retient la *première* promesse qui réussit, ignorant les échecs.
  - `Promise.race` : Accepte le résultat (succès ou échec) de la toute *première* promesse à franchir la ligne d'arrivée.

- **Syntaxe Moderne et Réseau** :
  - `async / await` : Transforme du code asynchrone complexe en instructions semblant s'exécuter ligne par ligne.
  - `Ajax` **&** `Fetch API` : `Fetch API` est l'outil standardisé basé sur les promesses pour récupérer ou envoyer des données (`GET`, `POST`) en arrière-plan (paradigme `Ajax`).

```
async function fetchData() {
  const response = await fetch("api/data");
  return await response.json();
}
```

  - `defer` : Attribut ordonnant au navigateur de télécharger un script en coulisse et de l'exécuter uniquement lorsque la page est entièrement affichée.

Niveau 9 : Gestion des Erreurs, Débogage et Mode Strict

Sécuriser l'exécution, analyser les failles et capturer les exceptions.

- **Mode Strict** :
  - `Strict Mode` **/** `use strict` : La consigne insérée en début de fichier exigeant que le langage soit rigoureux, interdisant les pratiques permissives ou obsolètes (comme utiliser une variable non déclarée).
- **Exceptions et Capture d'Erreurs** :
  - `throw` : Commande permettant de lancer volontairement une erreur.

    - `Exceptions` **(`try...catch...finally`)** : Bloc de protection. `try` essaie d'exécuter un processus volatil, `catch` capture l'erreur pour empêcher le programme de planter, et `finally` s'exécute toujours à la fin pour nettoyer la zone de travail.
- **Objets d'Erreur** :
    - `Error` : L'objet fondamental standardisant le message et la trace de l'erreur.
    - `DOMException` : Classe d'erreur spécifique au navigateur lorsque l'on manipule la page Web de manière interdite.
- `Debugger` : Un signal d'arrêt forçant le programme à se figer, permettant au développeur d'inspecter l'état actuel de la mémoire.

Niveau 10 : Prototypes et Mécanismes de Compilation

Les mécanismes d'héritage profonds et le fonctionnement interne du moteur.

- **L'Héritage par Prototype** :
    - `Objet à prototype` / `Prototype` : Le mécanisme intrinsèque de clonage où un objet peut emprunter les outils d'un autre objet parent via sa chaîne de prototypes.
    - `constructor` : La méthode d'initialisation qui s'exécute automatiquement lors de la création d'un nouvel objet pour configurer ses propriétés.
    - `instanceof` : Outil permettant de valider si un objet provient d'un modèle spécifique.
- **Performance du Moteur** :
    - `Interpreter / Compiler à la volée (JIT)` : Le moteur (comme `V8`) qui lit le code source et le traduit instantanément en langage machine au moment de l'exécution, optimisant drastiquement les routines les plus utilisées.

Niveau 11 : Environnements d'Exécution, API et Frameworks

Déploiement de JavaScript au-delà du navigateur et architectures applicatives à grande échelle.

- **Environnements d'Exécution Serveur** :
    - `Node.js` : L'environnement historique qui a libéré `JavaScript` du navigateur, permettant de créer de puissants serveurs et d'accéder au système de fichiers.
    - `Deno` : Le successeur moderne et sécurisé, verrouillant nativement les accès réseau.
    - `Bun` : Le runtime tout-en-un ultra-rapide remplaçant de multiples outils d'exécution et de compilation en un seul binaire.
- **Gestion des Dépendances** :
    - `NPM` **(Node Package Manager)** : L'immense catalogue mondial permettant de télécharger et d'intégrer des bibliothèques de code existantes à son propre projet.
- **Architecture à Grande Échelle** :
    - `API` **(Application Programming Interface)** : Les règles de communication standardisées permettant à différentes applications de s'échanger des données en toute sécurité.
    - `Framework` : Un squelette logiciel complet imposant une méthode de travail stricte pour construire l'application.
    - `Angular` : Le framework exhaustif développé par Google, incontournable pour les grandes applications d'entreprise (`SPA`).
    - `React Native / Ionic` : Des ponts technologiques traduisant le code Web (`JavaScript`) en applications mobiles natives pour iOS et Android.